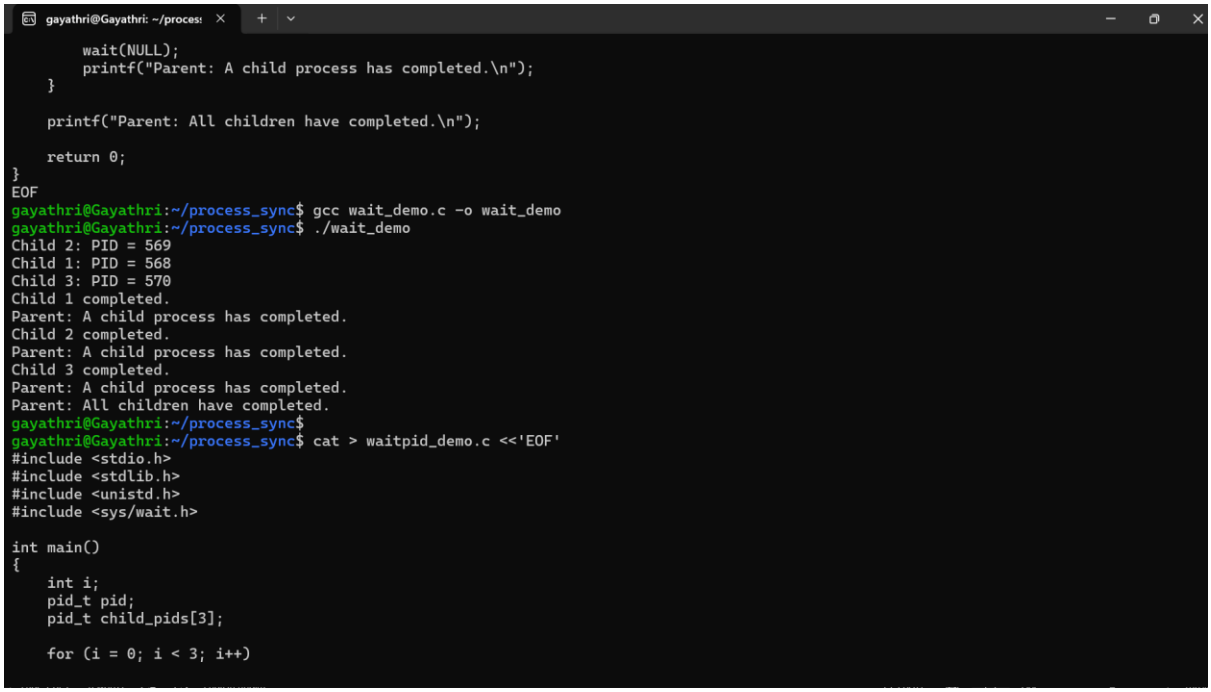
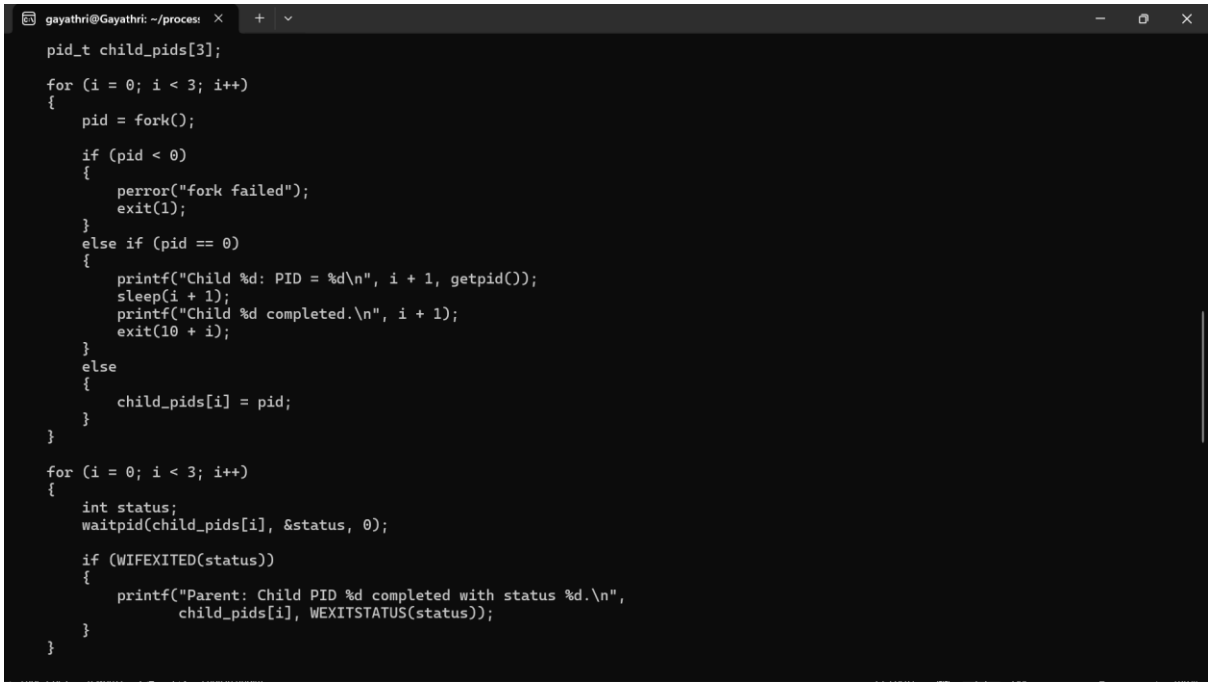
gayathri@Gayathri: ~/proces

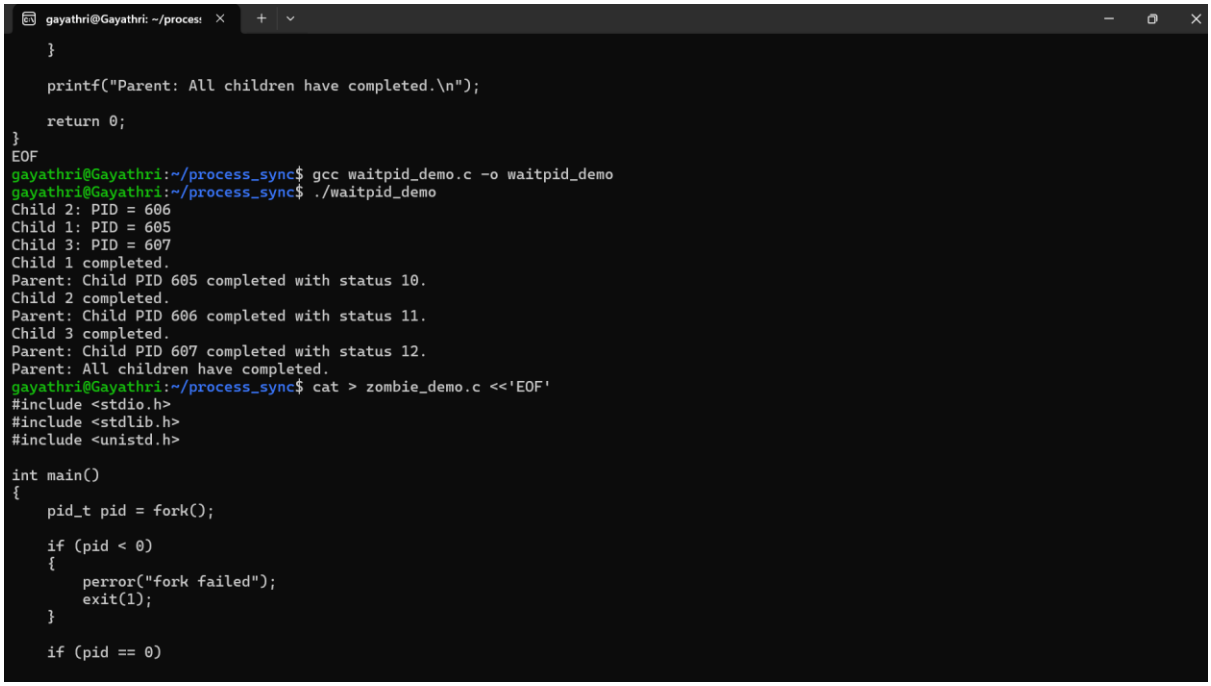
```
gayathri@Gayathri:~$ mkdir -p ~/process_sync
gayathri@Gayathri:~$ cd ~/process_sync
gayathri@Gayathri:~/process_sync$
gayathri@Gayathri:~/process_sync$ cat > wait_demo.c <<'EOF'
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    int i;
    pid_t pid;

    for (i = 0; i < 3; i++)
    {
        pid = fork();

        if (pid < 0)
        {
            perror("fork failed");
            exit(1);
        }
        else if (pid == 0)
        {
            printf("Child %d: PID = %d\n", i + 1, getpid());
            sleep(i + 1);
            printf("Child %d completed.\n", i + 1);
            exit(0);
        }
    }

    for (i = 0; i < 3; i++)
    {
        wait(NULL);
```



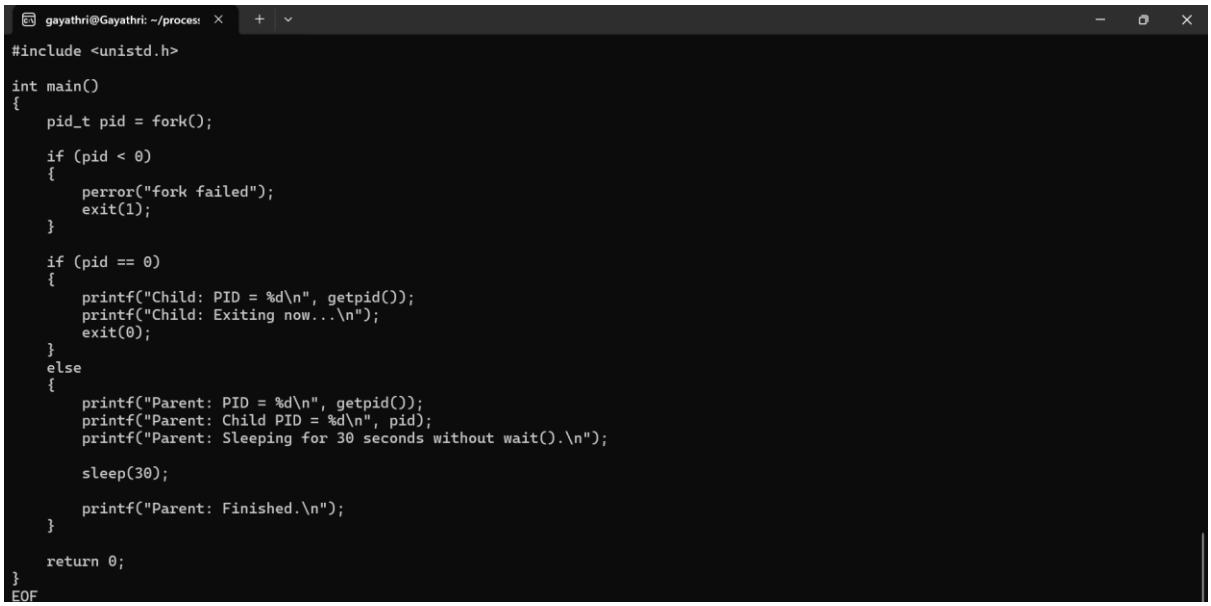
```
gayathri@Gayathri: ~/proces:

        wait(NULL);
        printf("Parent: A child process has completed.\n");
    }

    printf("Parent: All children have completed.\n");

    return 0;
}
EOF
gayathri@Gayathri:~/process_sync$ gcc wait_demo.c -o wait_demo
gayathri@Gayathri:~/process_sync$ ./wait_demo
Child 2: PID = 569
Child 1: PID = 568
Child 3: PID = 570
Child 1 completed.
Parent: A child process has completed.
Child 2 completed.
Parent: A child process has completed.
Child 3 completed.
Parent: A child process has completed.
Parent: All children have completed.
gayathri@Gayathri:~/process_sync$
gayathri@Gayathri:~/process_sync$ cat > waitpid_demo.c <<'EOF'
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    int i;
    pid_t pid;
    pid_t child_pids[3];

    for (i = 0; i < 3; i++)
```



```
gayathri@Gayathri: ~/proces:

    pid_t child_pids[3];

    for (i = 0; i < 3; i++)
    {
        pid = fork();

        if (pid < 0)
        {
            perror("fork failed");
            exit(1);
        }
        else if (pid == 0)
        {
            printf("Child %d: PID = %d\n", i + 1, getpid());
            sleep(i + 1);
            printf("Child %d completed.\n", i + 1);
            exit(10 + i);
        }
        else
        {
            child_pids[i] = pid;
        }
    }

    for (i = 0; i < 3; i++)
    {
        int status;
        waitpid(child_pids[i], &status, 0);

        if (WIFEXITED(status))
        {
            printf("Parent: Child PID %d completed with status %d.\n",
                   child_pids[i], WEXITSTATUS(status));
        }
    }
```



```
    }

    printf("Parent: All children have completed.\n");

    return 0;
}
EOF
gayathri@Gayathri:~/process_sync$ gcc waitpid_demo.c -o waitpid_demo
gayathri@Gayathri:~/process_sync$ ./waitpid_demo
Child 2: PID = 606
Child 1: PID = 605
Child 3: PID = 607
Child 1 completed.
Parent: Child PID 605 completed with status 10.
Child 2 completed.
Parent: Child PID 606 completed with status 11.
Child 3 completed.
Parent: Child PID 607 completed with status 12.
Parent: All children have completed.
gayathri@Gayathri:~/process_sync$ cat > zombie_demo.c <<'EOF'
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main()
{
    pid_t pid = fork();

    if (pid < 0)
    {
        perror("fork failed");
        exit(1);
    }

    if (pid == 0)
```



```
#include <unistd.h>

int main()
{
    pid_t pid = fork();

    if (pid < 0)
    {
        perror("fork failed");
        exit(1);
    }

    if (pid == 0)
    {
        printf("Child: PID = %d\n", getpid());
        printf("Child: Exiting now...\n");
        exit(0);
    }
    else
    {
        printf("Parent: PID = %d\n", getpid());
        printf("Parent: Child PID = %d\n", pid);
        printf("Parent: Sleeping for 30 seconds without wait().\n");

        sleep(30);

        printf("Parent: Finished.\n");
    }

    return 0;
}
EOF
```